

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 6

Operating Systems

Inside the xv6 File System

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment and evidence 3

2.2 Part 1 - What ls shows 3

2.3 Part 2 - mystat and hard links 4

2.4 Part 3 - Filesystem limits 5

2.5 Part 4 - Recursive find 6

2.6 Stretch - Show inode numbers 7

3 Conclusions **8**

A Submitted source code **9**

A.1 user/mystat.c 9

A.2 user/find.c 9

1 Introduction

Read the xv6 filesystem layout, implement readable stat output, derive its storage limits, and traverse nested directories using a recursive find utility.

This report follows the teacher's handout and includes every required Observe answer. Terminal-style listings replace new screenshots. Recorded output is identified with its execution environment; shortened excerpts are marked. Source files and build instructions are provided in the submission directory.

2 Tasks and results

2.1 Environment and evidence

Tested on 23 October 2026 with MIT xv6-riscv commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`, RISC-V GNU tools and QEMU. The recorded session used a combined image containing labs 04, 06 and 08, so executable sizes and assigned inode numbers can differ from the teacher's example or a minimal image. Each lab Makefile also built independently. `terminal-output.txt` contains the full shared test session. The source constants and line numbers below were read from the pinned checkout.

2.2 Part 1 - What ls shows

The four columns in xv6 `ls` are name, numeric type, inode number and logical size in bytes. They are different from Linux `ls -l` permissions/ownership columns.

2.2.1 1. Types 1, 2 and 3

`kernel/stat.h` defines `T_DIR = 1` (directory), `T_FILE = 2` (regular file), and `T_DEVICE = 3` (device). `console` has type 3; it accesses a device driver instead of ordinary file data blocks.

2.2.2 2. Root inode

The root inode is 1, set by `ROOTINO` in `kernel/fs.h`. Its `.` and `..` entries both name inode 1 because the root is its own parent. Representative recorded rows were:

```
.              1 1 1024
..             1 1 1024
README         2 2 2441
console        3 32 0
```

2.2.3 3. Empty directory size

```
$ mkdir a
$ ls a
.              1 35 32
..             1 1 1024
```

An entry is `sizeof(struct dirent) = 16` bytes: a two-byte inode number and a 14-byte name. A new empty directory contains `.` and `..`, so its logical size is `2 * 16 = 32` bytes. It can still

occupy a whole allocated filesystem block. An entry whose name uses all 14 bytes has no terminating NUL in the disk record.

2.3 Part 2 - mystat and hard links

`user/mystat.c` accepts one or more paths, calls `stat`, maps numeric types to readable names and prints inode, link count and byte size. A failed path reports to descriptor 2 and does not prevent later paths from being processed. A usage or stat error results in a nonzero exit status.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 32, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

2.3.1 1. What changed after ln and rm

Before linking, README had inode 2, size 2441 and one link. `ln README r2` makes both names resolve to inode 2, raises the shared link count to 2 and does not copy the data. Unlinking README removes that name and reduces the count to 1; r2's inode, type, size and contents remain unchanged. `cat r2 | wc` confirms that its 2441 bytes are still readable.

The root directory in this session already had a subdirectory from lab 04, so its link count was 2 rather than the initial image's 1. xv6 does not count `.` the same way as the Linux directory convention from lab 05; do not apply Linux's new-directory link count mechanically to xv6.

2.3.2 2. Where unlink decreases nlink and when storage is freed

At the pinned revision, `kernel/sysfile.c:244`, within `sys_unlink`, contains `ip->nlink--;`, followed by `iupdate(ip)` and `iunlockput(ip)`. The similarly spelled decrement in `sys_link` is error rollback, not the successful unlink path.

```
grep -n 'ip->nlink--' kernel/sysfile.c
sed -n '235,250p' kernel/sysfile.c
sed -n '345,380p' kernel/fs.c
```

The directory slot is cleared before that decrement. In `kernel/fs.c`, `iput` reclaims the inode when its link count is zero and the last in-memory reference is being released (`ref == 1` at entry). It calls `itrunc` to free direct and indirect data blocks and later makes the inode reusable. If a process still has the file open, its reference delays reclamation until close/exit releases it. These operations occur within filesystem transactions.

2.3.3 3. Fields of the on-disk inode

`struct dinode` contains `short type`, `short major`, `short minor`, `short nlink`, `uint size`, and `uint addrs[NDIRECT + 1]`. With this layout it occupies 64 bytes. `major` and `minor` identify a device driver/device number for device inodes; `addrs` stores 12 direct block addresses and one indirect address. The inode has no filename, owner UID, owner GID, access-mode bits or timestamps. This baseline has no symbolic-link type and no Unix permission checking. The separate in-memory inode adds bookkeeping such as reference counts and locking.

2.4 Part 3 - Filesystem limits

2.4.1 1. Long names and DIRSIZ

```
$ mkdir averyveryverylongname
$ ls
[excerpt]
averyveryveryl 1 36 32
```

`DIRSIZ = 14`, so the stored component is the first 14 characters, `averyveryveryl`. Two longer names with the same first 14 bytes can resolve to the same component. This is a fixed-size name limitation, not terminal clipping.

2.4.2 2. Largest file calculation

```
BSIZE = 1024 bytes
NDIRECT = 12 blocks
NINDIRECT = BSIZE / sizeof(uint) = 1024 / 4 = 256 blocks
MAXFILE = NDIRECT + NINDIRECT = 268 data blocks
Maximum content = (12 + 256) * 1024 = 274432 bytes = 268 KiB
```

The indirect block stores 256 block numbers, not file content, so it contributes addressing capacity but is not an extra data block in this sum. A maximum-size file also consumes an indirect metadata block, its inode and other filesystem metadata. The result is the per-file addressing limit; free space can impose a lower practical limit.

2.4.3 3. Disk layout from mkfs

Recorded during the actual build:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
    2000
```

Region	Blocks	Block indices
Boot and superblock	2	0 and 1
Log (header plus 30 payload blocks)	31	2 through 32

Region	Blocks	Block indices
Inodes	13	33 through 45
Free-block bitmap	1	46
Data region	1953	47 through 1999

The total is `2 + 31 + 13 + 1 + 1953 = 2000` blocks, or 2,048,000 bytes. The data-region size includes blocks already occupied by the initial directories and user programs; it is not a free-block count. Older xv6 revisions can print a 30-block log and 1954 data blocks; the recorded checkout reserves a separate log header and prints 31.

2.5 Part 4 - Recursive find

`user/find.c` walks from the supplied path and compares each last component using `strcmp`. It handles regular files and directories, skips free entries (`inum == 0`) and explicitly skips `.` and `..`. A 14-byte on-disk name is copied into a 15-byte buffer and NUL-terminated before comparison. Constructed paths are checked against `MAXPATH` and trailing slashes on the starting path are normalized.

To avoid consuming the 16-descriptor per-process limit with recursive depth, each directory entry read opens the directory, skips to that entry, reads it, and closes the descriptor before recursion. A shared path buffer also reduces stack use. This trades O(n squared) reads per directory for bounded descriptor use; it is adequate for the small xv6 tree. As with the basic exercise, the walk assumes directories are not concurrently modified. Paths and the one-page user stack still bound the maximum supported depth.

2.5.1 Nested-directory test

For an independent run, use a fresh image, as the handout requests. The combined recorded session already contained `a` and a hard link named `r2` from earlier experiments; it reused `a` and removed the old `r2` before linking `cat`.

```
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ rm r2
$ ln cat r2
```

```
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

The three matches have the same last component but different paths. The directory `a/b` is also a match when searching for `b`; the program is not restricted to regular files.

2.5.2 Error handling and trailing slash

```
$ mystat missing cat
mystat: cannot stat missing
cat: file, inode 3, links 2, size 36280 bytes
$ find no-such r2
find: cannot stat no-such
$ find a/ b
a/b
$ find a r2 -x
usage: find <dir> <name> [-n]
$ find
usage: find <dir> <name> [-n]
```

The stat error did not suppress the valid `cat` result. A nonexistent starting path gives an error rather than silence; a bad optional argument or missing arguments gives usage. `find a/ b` produces the same normalized result as `find a b`.

2.6 Stretch - Show inode numbers

```
$ find . r2 -n
./a/b/r2 inode 38
./a/r2 inode 39
./r2 inode 3
$ mystat cat r2
cat: file, inode 3, links 2, size 36280 bytes
r2: file, inode 3, links 2, size 36280 bytes
```

Only `./r2` has inode 3, the same as `cat`. The files `./a/b/r2` and `./a/r2` are independent files, inodes 38 and 39. Matching filenames do not imply shared content or inode identity; matching inode numbers on the same filesystem do.

3 Conclusions

The xv6 implementation makes inode identity and directory structure directly visible. mystat confirms that hard links share one inode and that unlink changes the link count rather than copying or immediately deleting data. The disk constants yield a 268 KiB maximum file and a 2000-block image. find locates nested matches and its optional inode display distinguishes equal names from actual hard links.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 6: *Inside the xv6 File System*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/mystat.c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

static char *
typename(int type)
{
  switch(type){
  case T_DIR: return "directory";
  case T_FILE: return "file";
  case T_DEVICE: return "device";
  default: return "unknown";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
  int failed = 0;
  if(argc < 2){
    fprintf(2, "usage: mystat <path> [path ...]\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      failed = 1;
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(failed);
}
```

A.2 user/find.c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "kernel/param.h"
#include "kernel/fcntl.h"
#include "user/user.h"

static char path[MAXPATH];
static char *wanted;
static int show_inode;
static int failed;

/* Read one entry and close before recursion: depth never consumes FDs.
 * Reopening and skipping costs O(n^2) reads per directory, acceptable here.
 * The directory must not be modified during the walk. */
static int
```

```
entry_at(int index, struct dirent *entry)
{
  int fd = open(path, O_RDONLY);
  if(fd < 0){
    fprintf(2, "find: cannot open %s\n", path);
    failed = 1;
    return -1;
  }
  int n = 0;
  for(int i = 0; i <= index; i++){
    n = read(fd, entry, sizeof(*entry));
    if(n != sizeof(*entry))
      break;
  }
  close(fd);
  if(n == 0)
    return 0;
  if(n != sizeof(*entry)){
    fprintf(2, "find: directory read failed: %s\n", path);
    failed = 1;
    return -1;
  }
  return 1;
}

static void
walk(void)
{
  struct stat st;
  struct dirent entry;
  char name[DIRSIZ + 1];
  if(stat(path, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    failed = 1;
    return;
  }
  char *base = path;
  for(char *p = path; *p; p++)
    if(*p == '/' && p[1])
      base = p + 1;
  if(strcmp(base, wanted) == 0){
    if(show_inode)
      printf("%s inode %d\n", path, st.ino);
    else
      printf("%s\n", path);
  }
  if(st.type != T_DIR)
    return;
  int len = strlen(path);
  for(int index = 0; entry_at(index, &entry) > 0; index++){
    if(entry.inum == 0)
      continue;
    memmove(name, entry.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;
    int slash = len > 0 && path[len - 1] != '/';
    if(len + slash + strlen(name) + 1 > sizeof(path)){
      fprintf(2, "find: path too long below %s\n", path);
      failed = 1;
      continue;
    }
    if(slash)
      path[len] = '/';
    strcpy(path + len + slash, name);
    walk();
```

```
    path[len] = 0;
  }
}

int
main(int argc, char *argv[])
{
  if((argc != 3 && argc != 4) ||
     (argc == 4 && strcmp(argv[3], "-n") != 0)){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  if(strlen(argv[1]) == 0 || strlen(argv[1]) >= sizeof(path)){
    fprintf(2, "find: invalid starting path\n");
    exit(1);
  }
  strcpy(path, argv[1]);
  int len = strlen(path);
  while(len > 1 && path[len - 1] == '/')
    path[--len] = 0;
  wanted = argv[2];
  show_inode = argc == 4;
  walk();
  exit(failed);
}
```